Interview Questions and Answers

1. **What is a Design Pattern? Why are they important in software development?**

Answer:
A design pattern is a reusable solution to a common problem in software design. It provides a template for solving problems that can be adapted to fit specific needs. They are important because they promote best practices, improve code maintainability, and help developers communicate solutions effectively.

2. **How are design patterns categorized?**

Answer:
Design patterns are typically categorized into three types:

- **Creational Patterns**: Focus on object creation (e.g., Singleton, Builder).
- **Structural Patterns**: Deal with object composition (e.g., Adapter, Composite).
- **Behavioral Patterns**: Address object interaction (e.g., Strategy, Observer).

3. **What is the difference between a Factory Method and an Abstract Factory?**

Answer:

- **Factory Method**: Creates objects using a single method in a class.
- **Abstract Factory**: Provides an interface to create families of related objects without specifying their concrete classes.

4. **What are some common pitfalls of using the Singleton pattern?**

Answer:

- Difficult to unit test due to hidden dependencies.
- Can introduce global state, leading to tight coupling.
- Thread-safety issues if not implemented correctly.

5. **What is the Open/Closed Principle, and how does it relate to design patterns?**

Answer:
The Open/Closed Principle states that software entities should be open for extension but closed for modification. Design patterns like Strategy and Decorator support this principle by allowing behaviors to be added dynamically without altering existing code.

6. **Write a Singleton class in C#.**

```
public sealed class Singleton
{
    private static readonly Lazy<Singleton> instance =
            new Lazy<Singleton>(() => new Singleton());

    private Singleton() {}
    public static Singleton Instance => instance.Value;
}
```

7. **What's wrong with this Singleton implementation?**

```
public class Singleton
{
    private static Singleton instance;
    private Singleton() { }
    public static Singleton GetInstance()
    {
        if (instance == null)
        {
            instance = new Singleton();
        }
        return instance;
    }
}
```

Answer:

- This implementation is **not thread-safe**. In a multithreaded environment, multiple threads could create separate instances.

8. **Implement the Strategy pattern to calculate discounts in C#.**

```
public interface IDiscountStrategy
{
    decimal ApplyDiscount(decimal price);
}
public class NoDiscount : IDiscountStrategy
{
    public decimal ApplyDiscount(decimal price) => price;
}
public class PercentageDiscount : IDiscountStrategy
{
    private readonly decimal percentage;
    public PercentageDiscount(decimal percentage) => this.percentage = percentage;
    public decimal ApplyDiscount(decimal price) => price - (price * percentage / 100);
}
public class DiscountContext
{
    private IDiscountStrategy strategy;
    public void SetStrategy(IDiscountStrategy strategy) => this.strategy = strategy;
    public decimal GetFinalPrice(decimal price) => strategy.ApplyDiscount(price);
}
```

9. **How would you refactor the following code using the Factory Method pattern?**

```
if (type == "Car")
    return new Car();
else if (type == "Truck")
    return new Truck();
```

Answer:

```
public abstract class VehicleFactory
{
    public abstract IVehicle CreateVehicle();
}
public class CarFactory : VehicleFactory
{
    public override IVehicle CreateVehicle() => new Car();
}
public class TruckFactory : VehicleFactory
{
    public override IVehicle CreateVehicle() => new Truck();
}
```

10. **Implement a simple Observer pattern to notify when a stock price changes.**

```
public interface IObserver
{
    void Update(decimal price);
}
public class Stock
{
    private readonly List<IObserver> observers = new();
    private decimal price;
    public void Attach(IObserver observer) => observers.Add(observer);
    public void Detach(IObserver observer) => observers.Remove(observer);
    public void Notify() => observers.ForEach(o => o.Update(price));
    public decimal Price
    {
        get => price;
        set { price = value; Notify(); }
    }
}
public class StockObserver : IObserver
{
    public void Update(decimal price) => Console.WriteLine($"Stock price updated to {price}");
}
```

11. **Why is the Builder pattern better than telescoping constructors?**

Answer:
The Builder pattern avoids constructor overloading by separating the construction process from the object itself, making the code easier to read and maintain.

12. **When would you use the Adapter pattern?**

Answer:
When you need to bridge incompatible interfaces to make them work together without changing their implementations.

13. **What is the main advantage of the Composite pattern?**

Answer:
The Composite pattern allows treating individual objects and compositions of objects uniformly, enabling hierarchical structures like trees.

14. **How does the Decorator pattern differ from inheritance?**

Answer:
The Decorator pattern adds responsibilities dynamically to an object, while inheritance adds behavior at compile time, potentially leading to a rigid class hierarchy.

15. **Can a class be its own Observer? Explain with an example.**

Answer:
Yes, a class can observe itself if it implements both the Subject and Observer roles. For example, a class managing user data could update its own state based on changes to its properties.

16. **How would you use the Command pattern to implement an undo/redo system?**

Answer:
Store each executed command in a stack. For undo, reverse the operation of the top command. For redo, re-execute the undone command stored in another stack.

17. **How can the Chain of Responsibility pattern improve logging in an application?**

Answer:
Different log handlers (e.g., File, Console, Remote Server) can be chained together, passing log messages down the chain until one handles it.

18. **What is double dispatch, and which design pattern uses it?**

Answer:
Double dispatch allows an operation to be determined by the runtime types of two objects. The Visitor pattern uses this technique to separate algorithms from object structures.

19. **What are the trade-offs of using the Proxy pattern for security checks?**

Answer:
Pros: Centralized control and simplified client access.
Cons: Adds a level of indirection, which might affect performance.

20. **How would you combine the Factory Method and Strategy patterns in a real-world scenario?**

Answer:
Use the Factory Method to create different Strategy implementations dynamically based on runtime input (e.g., selecting a sorting algorithm or a payment processor).

21. **What are the SOLID principles, and how do they relate to design patterns?**

Answer:
SOLID principles are guidelines for writing clean, maintainable code. Design patterns often adhere to these principles to provide scalable solutions. For example, the Strategy pattern aligns with the Open/Closed Principle.

22. **What is the purpose of the Template Method pattern?**

Answer:
It defines the skeleton of an algorithm in a base class, allowing subclasses to implement specific steps while keeping the algorithm structure intact.

23. **How does the Prototype pattern handle object creation?**

Answer:
The Prototype pattern creates objects by cloning an existing instance, often to avoid costly instantiation of complex objects.

24. **What is the difference between the Decorator and Proxy patterns?**

Answer:

- **Decorator**: Dynamically adds behavior to an object without affecting others.
- **Proxy**: Acts as a surrogate or placeholder for another object, controlling access to it.

25. **What is the primary use case for the Command pattern?**

Answer:
The Command pattern encapsulates a request as an object, allowing parameterization of objects with different requests, queuing, or logging requests, and supporting undoable operations.

26. **What problem does the Mediator pattern solve?**

Answer:
It centralizes complex communications and control logic between objects, reducing dependencies and promoting loose coupling.

27. **Why is the Singleton pattern considered an anti-pattern in some cases?**

Answer:
Singleton introduces global state, making the code harder to test and maintain. It can also lead to tight coupling and violate the Single Responsibility Principle.

28. **What is lazy initialization in the context of design patterns?**

Answer:
Lazy initialization is a technique where an object is created only when it is needed, commonly used in the Singleton and Proxy patterns.

29. **How does the Chain of Responsibility pattern handle requests?**

Answer:
It passes requests along a chain of handlers, where each handler decides to process or pass the request to the next handler.

30. **What is the role of the Flyweight pattern?**

Answer:
The Flyweight pattern minimizes memory usage by sharing as much data as possible with similar objects, useful in scenarios with large numbers of fine-grained objects.

31. **Write a simple Factory Method to create a database connection in C#.**

```
public abstract class DatabaseFactory
{
    public abstract IDatabase CreateDatabase();
}
public class SqlDatabaseFactory : DatabaseFactory
{
    public override IDatabase CreateDatabase() => new SqlDatabase();
}
public class NoSqlDatabaseFactory : DatabaseFactory
{
    public override IDatabase CreateDatabase() => new NoSqlDatabase();
}
```

32. **Refactor this code using the State pattern.**

```
if (status == "New")
    Console.WriteLine("Order is New");
else if (status == "Shipped")
    Console.WriteLine("Order is Shipped");
```

Answer:

```
public interface IOrderState
{
    void HandleOrder();
}
public class NewState : IOrderState
{
    public void HandleOrder() => Console.WriteLine("Order is New");
}
public class ShippedState : IOrderState
{
    public void HandleOrder() => Console.WriteLine("Order is Shipped");
}
```

33. **Implement a Composite pattern to manage an organization tree.**

```
public interface IEmployee
{
    void ShowDetails();
}
public class Developer : IEmployee
{
    private string name;
    public Developer(string name) => this.name = name;
    public void ShowDetails() => Console.WriteLine($"Developer: {name}");
}
public class Manager : IEmployee
{
    private List<IEmployee> subordinates = new();
    public void AddSubordinate(IEmployee employee) => subordinates.Add(employee);
    public void ShowDetails()
    {
        Console.WriteLine("Manager:");
        subordinates.ForEach(e => e.ShowDetails());
    }
}
```

34. **How would you implement undo functionality using the Memento pattern in C#?**

```
public class EditorState
{
    public string Content { get; }
    public EditorState(string content) => Content = content;
}
public class Editor
{
    public string Content { get; set; }
    public EditorState SaveState() => new EditorState(Content);
    public void RestoreState(EditorState state) => Content = state.Content;
}
```

35. **Write code for a simple Adapter pattern for incompatible interfaces.**

```
public interface ITarget
{
    void Process();
}
public class Adaptee
{
    public void SpecificRequest() => Console.WriteLine("Specific Request");
}
public class Adapter : ITarget
{
    private Adaptee adaptee = new();
    public void Process() => adaptee.SpecificRequest();
}
```

36. **When would you use the Bridge pattern?**

Answer:
When you want to separate an object's abstraction from its implementation so that both can vary independently.

37. **How does the Visitor pattern adhere to the Open/Closed Principle?**

Answer:
It allows adding new operations to existing object structures without modifying the objects themselves by separating the algorithm from the object.

38. **What is the main difference between the Builder and Abstract Factory patterns?**

Answer:

- Builder focuses on constructing complex objects step by step.
- Abstract Factory creates families of related objects without detailing their concrete classes.

39. **What problem does the Iterator pattern solve?**

Answer:
It provides a way to access elements of a collection sequentially without exposing the underlying representation.

40. **How does the Observer pattern help decouple code?**

Answer:
It separates the subject from its observers, allowing them to be independently developed and modified without tightly coupling the two.

41. **What is the main purpose of the Dependency Injection (DI) pattern?**

Answer:
DI helps achieve loose coupling between objects by injecting dependencies at runtime instead of hard-coding them. This makes the application easier to test and maintain.

42. **What is the difference between Inversion of Control (IoC) and Dependency Injection (DI)?**

Answer:
IoC is a broader principle where control of object creation and flow is inverted (e.g., handed to a container). DI is a specific implementation of IoC that involves passing dependencies to a class.

43. **How do you prevent breaking the Liskov Substitution Principle when using design patterns?**

Answer:
Ensure derived classes can be substituted for their base class without altering the expected behavior. Use patterns like Strategy or State to encapsulate behavior instead of overriding methods improperly.

44. **What is a "Pure Fabrication" design principle, and how does it relate to design patterns?**

Answer:
Pure Fabrication suggests creating classes or patterns not derived from the domain model to achieve better separation of concerns. Examples include Repository and Service classes.

45. **What is the main disadvantage of the Abstract Factory pattern?**

Answer:
It can lead to a complex hierarchy of factory classes, which may increase code maintenance effort if there are too many product families.

46. **How does the Visitor pattern differ from the Command pattern?**

Answer:

- **Visitor**: Focuses on applying new operations to an object structure without modifying the structure itself.
- **Command**: Encapsulates requests as objects to support undo/redo and queueing.

47. **What is the main purpose of the Builder pattern in the context of immutability?**

Answer:
The Builder pattern allows constructing immutable objects with many optional properties, avoiding the need for a constructor with numerous parameters.

48. **How do you ensure thread safety in a Singleton implementation in C#?**

Answer:
Use a Lazy<T> initialization or double-checked locking:

```
public sealed class Singleton
{
    private static readonly Lazy<Singleton> instance = new Lazy<Singleton>(() => new Singleton());
    private Singleton() { }
    public static Singleton Instance => instance.Value;
}
```

49. **What are the trade-offs between Prototype and Flyweight patterns?**

Answer:

- **Prototype**: Focuses on creating new objects by copying existing ones.
- **Flyweight**: Reduces memory usage by sharing immutable data among objects. The trade-off is between creation speed (Prototype) and memory efficiency (Flyweight).

50. **Can the Bridge pattern and Adapter pattern be used together? Explain.**

Answer:
Yes, the Adapter pattern can bridge the gap between the abstraction and implementation layers of the Bridge pattern, allowing for more flexibility in combining existing and new functionality.

51. **How would you refactor a monolithic application to a microservices architecture using design patterns?**

Answer:

- Use **Facade** to simplify the existing API.
- Apply **Event-Driven Architecture** using patterns like **Observer** or **Mediator**.
- Implement **Singleton** for shared configurations.

52. **What design pattern would you use to limit API calls and why?**

Answer:
The **Proxy** pattern can be used to add rate-limiting logic, ensuring that calls to the underlying service are restricted or queued.

53. **How would you handle caching in an application using design patterns?**

Answer:
Use the **Decorator** pattern to wrap the primary service logic with caching behavior, ensuring separation of concerns.

54. **Explain a real-world scenario where you would use the Chain of Responsibility pattern.**

Answer:
In a technical support system, a request could pass through different levels of support (L1, L2, L3) until resolved.

55. **How would you use the Command pattern to implement retry logic?**

Answer:
Encapsulate the operation in a command object. If the operation fails, re-execute the command based on retry conditions.

56. **What is the most common mistake when implementing the Observer pattern?**

Answer:
Not properly detaching observers, leading to memory leaks when observers are no longer needed but still receive updates.

57. **What are some anti-patterns related to Singleton?**

Answer:

- **Monostate Pattern**: Using static variables to simulate Singleton behavior.
- **Overuse**: Turning Singleton into a global variable for everything, violating the Single Responsibility Principle.

58. **When should you avoid using design patterns?**

Answer:
Avoid using design patterns when they add unnecessary complexity or when the problem is straightforward and doesn't benefit from abstraction.

59. **What is the "pattern fever," and how can it be avoided?**

Answer:
Pattern fever is overusing or misapplying design patterns. Avoid it by focusing on solving the actual problem first and applying patterns only when needed.

60. **How can the Open/Closed Principle be violated when implementing a Factory Method?**

Answer:
By directly modifying the Factory Method class to handle new product types instead of creating new subclasses, leading to tight coupling and reduced scalability.

61. **What is the Template Method pattern's key principle?**

Answer:
It defines the steps of an algorithm in a method, deferring some steps to subclasses. This allows the structure of the algorithm to remain unchanged while letting subclasses define specific behavior.

62. **Why is the Strategy pattern preferred over conditionals for algorithms?**

Answer:
It avoids large if-else or switch statements, promotes the Open/Closed Principle, and allows behavior changes dynamically by swapping strategies at runtime.

63. **What is the main goal of the Bridge pattern?**

Answer:
To decouple abstraction from implementation, enabling them to vary independently. This is particularly useful when dealing with multi-dimensional changes (e.g., devices and platforms).

64. **What is the Factory Method's relationship to the Open/Closed Principle?**

Answer:
Factory Method supports the Open/Closed Principle by allowing the addition of new product types via subclassing instead of modifying existing code.

65. **What are the key participants in the Observer pattern?**

Answer:

- **Subject**: Maintains a list of observers and notifies them of changes.
- **Observers**: Implement an interface to receive updates from the subject.

66. **Write a simple implementation of the Decorator pattern to add features to a car.**

```
public interface ICar
{
    string GetFeatures();
}
public class BasicCar : ICar
{
    public string GetFeatures() => "Basic Car";
}
public class SunroofDecorator : ICar
{
    private readonly ICar car;
    public SunroofDecorator(ICar car) => this.car = car;
    public string GetFeatures() => $"{car.GetFeatures()} + Sunroof";
}
```

67. **How would you implement the Proxy pattern for a remote database?**

```
public interface IDatabase
{
    string GetData();
}
public class RealDatabase : IDatabase
{
    public string GetData() => "Data from remote database.";
}
public class DatabaseProxy : IDatabase
{
    private RealDatabase realDatabase;
    public string GetData()
    {
        if (realDatabase == null)
            realDatabase = new RealDatabase();
        return realDatabase.GetData();
    }
}
```

68. **Write a code example of the Chain of Responsibility pattern for request validation.**

```
public abstract class Handler
{
    protected Handler next;
    public void SetNext(Handler next) => this.next = next;
    public abstract void HandleRequest(string request);
}
public class AuthenticationHandler : Handler
{
    public override void HandleRequest(string request)
    {
        if (request.Contains("Auth"))
            Console.WriteLine("Authentication passed");
        else if (next != null)
            next.HandleRequest(request);
    }
}
```

69. **Demonstrate how the Composite pattern could represent a file system.**

```
public interface IFileSystemComponent
{
    void Display();
}
public class File : IFileSystemComponent
{
    private string name;
    public File(string name) => this.name = name;
    public void Display() => Console.WriteLine($"File: {name}");
}
public class Folder : IFileSystemComponent
{
    private string name;
    private List<IFileSystemComponent> children = new();
    public Folder(string name) => this.name = name;
    public void Add(IFileSystemComponent component) => children.Add(component);
    public void Display()
    {
        Console.WriteLine($"Folder: {name}");
        foreach (var child in children) child.Display();
    }
}
```

70. **Create an example of the Builder pattern for building an email.**

```
public class Email
{
    public string To { get; set; }
    public string Subject { get; set; }
    public string Body { get; set; }
}
public class EmailBuilder
{
    private Email email = new();
    public EmailBuilder SetTo(string to) { email.To = to; return this; }
    public EmailBuilder SetSubject(string subject) { email.Subject = subject; return this; }
    public EmailBuilder SetBody(string body) { email.Body = body; return this; }
    public Email Build() => email;
}
```

71. **How can you implement undo functionality using the Command pattern?**

Answer:
Store executed commands in a stack. When undoing, call the Unexecute method of the most recent command. For redo, re-execute the last undone command.

72. **When would you use the Flyweight pattern in a graphics application?**

Answer:
When rendering a large number of similar objects, such as characters in a text editor or trees in a forest, to minimize memory usage.

73. **How does the Mediator pattern improve communication in complex systems?**

Answer:
It centralizes control logic, reducing dependencies between objects and making the system easier to extend and maintain.

74. **What design pattern would you use for implementing a plugin system?**

Answer:
The **Factory Method** or **Abstract Factory** pattern can dynamically load and create plugin objects at runtime based on configuration or user input.

75. **How would you prevent excessive object creation in a performance-critical application?**

Answer:
Use the Flyweight pattern to share objects that have common, immutable state. For example, reuse font glyphs in a text-rendering system.

76. **How does the Strategy pattern differ from the State pattern?**

Answer:

- Strategy: Focuses on selecting an algorithm at runtime.
- State: Focuses on changing an object's behavior based on its internal state.

77. **How do design patterns help in applying the DRY principle?**

Answer:
Patterns like **Decorator** and **Template Method** allow reusable behavior without code duplication by sharing logic across multiple classes.

78. **What design pattern would you use for implementing object pooling?**

Answer:
The **Object Pool** pattern creates a pool of pre-instantiated objects that can be reused, reducing the overhead of frequent object creation and destruction.

79. **Why is the Repository pattern not considered part of the GoF design patterns?**
Answer:
The Repository pattern is part of domain-driven design (DDD) and focuses on data access, which is outside the scope of the GoF design patterns.

80. **What is the main drawback of overusing design patterns?**

Answer:
Overengineering: Introducing unnecessary abstraction can make the codebase overly complex, harder to maintain, and less performant.

81. **What are the key differences between the Adapter and Decorator patterns?**

Answer:

- **Adapter**: Changes the interface of an object to match what a client expects.
- **Decorator**: Enhances an object's behavior without altering its interface.

82. **What problem does the Null Object pattern solve?**

Answer:
The Null Object pattern avoids null checks by providing a default, do-nothing implementation of an interface or class.

83. **How does the Facade pattern simplify complex systems?**

Answer:
It provides a unified interface to a set of interfaces in a subsystem, making the subsystem easier to use by hiding its complexity.

84. **What is the purpose of the Marker Interface pattern?**

Answer:
A marker interface has no methods or fields but is used to convey metadata or behavior (e.g., Serializable in Java).

85. **How does the Singleton pattern violate the Single Responsibility Principle?**

Answer:
A Singleton often combines responsibilities of instance control and global access, which are two separate concerns.

86. **Implement a simple Facade pattern to abstract a complex order processing system.**

```
public class InventorySystem
{
    public void CheckStock(string item) => Console.WriteLine($"Checking stock for {item}...");
}
public class PaymentSystem
{
    public void ProcessPayment(string paymentMethod) => Console.WriteLine($"Processing payment via {paymentMethod}...");
}
public class ShippingSystem
{
    public void ShipItem(string item) => Console.WriteLine($"Shipping {item}...");
}
public class OrderFacade
{
    private InventorySystem inventory = new();
    private PaymentSystem payment = new();
    private ShippingSystem shipping = new();
    public void PlaceOrder(string item, string paymentMethod)
    {
        inventory.CheckStock(item);
        payment.ProcessPayment(paymentMethod);
        shipping.ShipItem(item);
    }
}
```

87. **Write a Proxy pattern to handle access control for a secure system.**

```
public interface ISecureSystem
{
    void AccessSystem(string user);
}
public class SecureSystem : ISecureSystem
{
    public void AccessSystem(string user) => Console.WriteLine($"{user} has accessed the system.");
}
public class SecureSystemProxy : ISecureSystem
{
    private SecureSystem secureSystem = new();
    private List<string> authorizedUsers = new() { "Admin", "Manager" };
    public void AccessSystem(string user)
    {
        if (authorizedUsers.Contains(user))
            secureSystem.AccessSystem(user);
        else
            Console.WriteLine($"Access denied for {user}.");
    }
}
```

88. **Refactor this code using the Template Method pattern:**

```
public void ProcessOrder(string type)
{
    if (type == "Online")
        Console.WriteLine("Processing online order...");
    else if (type == "InStore")
        Console.WriteLine("Processing in-store order...");
}
```

Answer:

```
public abstract class OrderProcessor
{
    public void ProcessOrder() => Console.WriteLine($"Order Type: {GetOrderType()}");
    protected abstract string GetOrderType();
}
public class OnlineOrder : OrderProcessor
{
    protected override string GetOrderType() => "Online";
}
public class InStoreOrder : OrderProcessor
{
    protected override string GetOrderType() => "In-Store";
}
```

89. **Demonstrate the Observer pattern with a stock price tracker.**

```
public interface IObserver
{
    void Update(decimal price);
}
public class StockTracker
{
    private List<IObserver> observers = new();
    private decimal stockPrice;
    public void AddObserver(IObserver observer) => observers.Add(observer);
    public void RemoveObserver(IObserver observer) => observers.Remove(observer);
    public void NotifyObservers() => observers.ForEach(o => o.Update(stockPrice));
    public decimal StockPrice
    {
        get => stockPrice;
        set { stockPrice = value; NotifyObservers(); }
    }
}
public class StockInvestor : IObserver
{
    public void Update(decimal price) => Console.WriteLine($"Stock price updated to {price}");
}
```

90. **Write a Builder pattern for configuring a gaming PC.**

```
public class GamingPC
{
    public string CPU { get; set; }
    public string GPU { get; set; }
    public int RAM { get; set; }
}
public class GamingPCBuilder
{
    private GamingPC pc = new();
    public GamingPCBuilder AddCPU(string cpu) { pc.CPU = cpu; return this; }
    public GamingPCBuilder AddGPU(string gpu) { pc.GPU = gpu; return this; }
    public GamingPCBuilder AddRAM(int ram) { pc.RAM = ram; return this; }
    public GamingPC Build() => pc;
}
```

91. **How would you implement a plugin architecture using design patterns?**

Answer:
Use the **Factory Method** or **Abstract Factory** pattern to dynamically create plugin objects. Use the **Adapter** pattern to ensure compatibility with the application.

92. **When would you use the Mediator pattern in a UI application?**

Answer:
Use it to manage interactions between components like buttons, text boxes, and dropdowns to avoid direct dependencies and simplify communication.

93. **How would you manage database transactions using the Command pattern?**

Answer:
Each database operation (Insert, Update, Delete) can be encapsulated as a Command object. A transaction manager can execute or roll back these commands as needed.

94. **What design pattern would you use to implement retries for failed API calls?**

Answer:
The **Command** pattern encapsulates the request, making it easier to retry upon failure.

95. **How would you use the State pattern in a vending machine?**

Answer:
Represent states like Idle, WaitingForPayment, and Dispensing as separate classes. Switch states based on user actions.

96. **What are common pitfalls when using the Singleton pattern?**

Answer:

- Thread-safety issues.
- Hidden dependencies making unit testing difficult.
- Global state leading to tight coupling.

97. **Why is the Abstract Factory pattern sometimes criticized?**

Answer:
It can lead to excessive complexity with too many factory classes, especially if the number of product families grows.

98. **What are the dangers of overusing the Flyweight pattern?**

Answer:
Overuse can lead to complexity in managing shared state versus intrinsic state and can introduce hard-to-debug issues.

99. **Why might the Observer pattern cause performance issues?**

Answer:
If there are many observers or frequent notifications, performance can degrade due to excessive updates or notifications.

100. **What is the most common mistake when using the Command pattern?**

Answer:
Tight coupling between Command objects and their receivers, which violates the goal of decoupling request and execution.